

**A
PROJECT REPORT
on**

“Whether App”

Submitted to:

*Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur for
Partial Fulfillment of the Degree of*

Master in Computer Application

Submitted by

**Ms. Achal Bais
Ms. Kalyani Hemane**

Under the Guidance of

**Prof. Shambhavi Holay
Prof. Nikita Khanzode**



Name of Department

NAAC Accredited with A+ Grade

ISO 9001:2015 Certified

Vidarbha Bahu-uddeshiya Shikshan Sanstha's
**Tulsiramji Gaikwad-Patil College of Engineering
& Technology, Nagpur-441108**
(An Autonomous Institute affiliated to RTMNU, Nagpur)

Session 2024-25



Tulsiramji Gaikwad-Patil College of Engineering & Technology, Nagpur-441108

(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj
Nagpur University, Nagpur)

CERTIFICATE

This is to certify that project work described in this report entitled, “**Whether App**” was carried out by **Ms. Achal A Bais and Ms. Kalyani Hemane** in Tulsiramji Gaikwad-Patil College of Engineering & Technology, Nagpur under my supervision and guidance in partial fulfillment of the requirement for the degree of Master in Computer Application of Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur.

This work is the own work of the candidate, completed in all respect and is of sufficiently high standard to warrant its submission to the said degree. The assistance and resources used for this work are duly acknowledged.

Name of Guide

**Prof. Shambhavi Holay
Prof. Nikita Khanzode**

Prof. T. P. Raju

Project Coordinator (MCA)

**Prof. Roshan A. Chandekar
HoD (MCA)**

**Dr. Pratik Ghutke
Dean Academics**

**Dr. Pragati Patil
Vice-Principal**

**Dr. P. L. Naktode
Principal**

**Name & Signature of
External Examiner**

**Name & Signature of
Internal Examiner**



**Tulsiramji Gaikwad-Patil College
of Engineering & Technology,
Nagpur-441108**



**(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj
Nagpur University, Nagpur)**

DECLARATION

I hereby declare that this project titled **“Whether App”** is a bonafide and authentic record of the work done by me under supervision of **Prof. Shambhavi Holay and Prof. Nikita khanzode** during academic session 2024-25.

The work presented here is not duplicated from any other source and also not submitted earlier for any other degree/diploma of any university. I understand that any such duplication is liable to be punished in accordance with the university rules. The source material, data used in this research have been duly acknowledged.

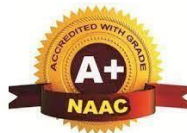
Date:

Place: Nagpur

Name and Signature of Student

Achal Bais

Kalyani Hemane



Tulsiramji Gaikwad-Patil College of Engineering & Technology, Nagpur-441108

**(An Autonomous Institute affiliated to Rashtrasant Tukadoji
Maharaj Nagpur University, Nagpur)**

ACKNOWLEDGEMENT

With profound feeling of immense gratitude and affection, I would like to thank my guide **Prof. Shambhavi Holay, Prof. Nikita Khanzode, MCA Department**, for his continuous support, motivation, enthusiasm and guidance. His encouragement, supervision with constructive criticism and confidence enabled me to complete this project.

I also wish to extend my reverences to **HOD Prof. Roshan Chandekar, Master in Computer Application Department** for providing necessary facilities to complete my project.

I am also thankful to all the faculty members and all non-teaching staff of the department & college for their cooperation throughout the project work.

I also put forth my/our deepest sense of gratitude towards the Principal, TGPCET for constant motivation and providing necessary infrastructure.

Achal Bais

Kalyani Hemane

PROJECTEE

Date:

Place:

CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Award for project presentation	v
Certificate(s) for project presentation	vi

CHAPTER – I

1. Introduction	1-5
1.1 Introduction to Weather App	2
1.2 Methodology used	3
1.3 Technologies used	4

CHAPTER – II

2. Software Requirement Specification	6-11
2.1 Problem Statement	7
2.2 Project Scope	7
2.3 Design and Implementation Constraint	7
2.4 User Documentation	8
2.5 Assumptions and Dependencies	8
2.6 System Features	8
2.6.1 Actors	
2.7 Functional Requirements	9
2.7.1 Accessing a database	
2.7.2 Prediction algorithm	
2.7.3 Actions performed by system	
2.8 Non-Functional Requirements	10
2.8.1 User Non Functional Requirements	
2.8.2 System Non Functional Requirements	
2.8.3 Other Non-Functional Requirements	

2.9 Other Requirements	10
2.9.1 Performance Requirements	
2.9.2 Safety Requirements	
2.9.3 Security Requirements	
2.9.4 Hardware Requirements	
2.9.5 Software Requirements	

CHAPTER – III

3 Design	12-14
-----------------	--------------

3.1 Introduction	
3.1.1 Proposed approach	
3.1.2 Use case Diagram	
3.1.3 UI Design	

CHAPTER – IV

4 Testing.	15-16
-------------------	--------------

4.1 Unit testing	
4.2 Integration Testing	

CHAPTER – V

5 Results	17-19
------------------	--------------

CHAPTER – VI

6 Conclusion & Future Scope	20-21
--	--------------

APPENDIX	22-27
-----------------	--------------

REFERENCES	28
-------------------	-----------

CHAPTER- I

Introduction

CHAPTER – I

Introduction

1.1 Introduction to Weather Forecasting

In today's fast-paced, technology-driven world, accurate and accessible weather prediction has become an indispensable tool for individuals, businesses, and governments. Weather conditions directly affect a wide range of activities—such as agriculture, aviation, tourism, event planning, and daily commuting—making timely access to weather information more important than ever. The availability of digital tools and web-based platforms has transformed how users interact with weather data, shifting the focus from traditional television and radio forecasts to real-time, user-centric applications available at their fingertips.

With the increasing demand for quick and accurate weather updates, web applications that provide location-based weather information have gained significant popularity. These apps are not only used by common users but also by professionals in logistics, disaster management, and urban planning. They serve as vital decision-making aids by helping users adapt to changing environmental conditions, manage risks, and ensure safety and productivity.

This project titled "Weather App " aims to develop a lightweight, browser-based application that enables users to check real-time weather updates for any city around the world. The app utilizes the OpenWeatherMap API, a widely-used third-party service that provides weather data in structured JSON format. Through this API, the application fetches up-to-date information such as temperature, humidity, wind speed, and general weather conditions, and visually presents this data using descriptive icons and readable formats.

The development focuses on the core web technologies of **HTML5**, **CSS3**, and **Vanilla JavaScript**, making the application fully client-side without relying on any server-side processing. This approach makes it easier for beginners and learners to understand how front-end components and APIs can work together to deliver dynamic content. The use of **asynchronous JavaScript** via the `fetch()` API allows the app to retrieve and display data without reloading the page, improving user experience and responsiveness.

The user interface is designed with simplicity and clarity in mind. A clean layout with intuitive input fields and result displays ensures that users can easily access the information they need. The app is made responsive using CSS media queries, allowing it to function effectively across a range of devices—from desktop browsers to smartphones

1.1 Methodology used

In a developing country and an economy like India where major population is dependent on agriculture, weather conditions play an important and vital role in economic growth of the overall nation. So, weather prediction should be more precise and accurate. Weather parameters are collected from the open source . Fig. 1.1 visualizes the system in the form of a block diagram.

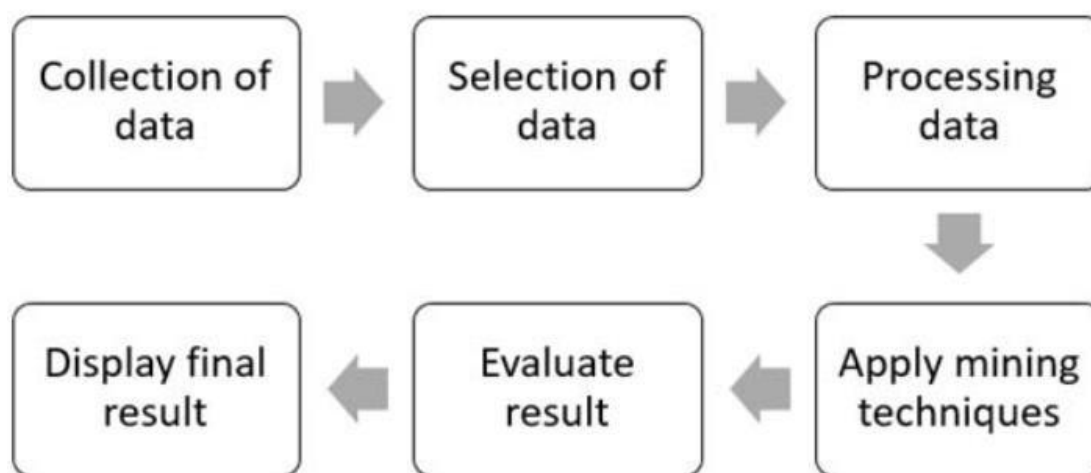


Fig 1.1 System Block Diagram

The weather is predicted using various indices like temperature, humidity. Temperature is the measure of hotness or coldness, generally measured using thermometer. Units of temperature most frequently used are Celsius and Fahrenheit. We have used maximum and minimum temperature values along with normal temperature as different index values for prediction of the weather.

Humidity is the quantity of water vapor present in the atmosphere. It is a relative quantity.

Dew point is the temperature of the atmosphere (which varies according to pressure and humidity) below which water droplets begin to condense and dew is formed.

1.2 Technologies Used

HTML

Hypertext Markup Language (HTML) is the standard markup language for documents designed to be displayed in a web browser. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript. Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

CSS

Cascading Style Sheets (CSS) is a style sheet language used for describing the presentation of a document written in a markup language like HTML. CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript. CSS is designed to enable the separation of presentation and content, including layout, colors, and fonts. This separation can improve content accessibility, provide more flexibility and control in the specification of presentation characteristics, enable multiple web pages to share formatting by specifying the relevant CSS in a separate .CSS file, and reduce complexity and repetition in the structural content.

JAVA SCRIPT

JavaScript often abbreviated as JS, is a high-level, interpreted scripting language that conforms to the ECMA Script specification. JavaScript has curlybracket syntax, dynamic typing, prototype-based object- orientation, and firstclass functions. Alongside HTML and CSS, JavaScript is one of the core technologies of the World Wide Web. JavaScript enables interactive web pages and is an essential part of web 6 applications. The vast majority of websites use it, and major web browsers have a dedicated JavaScript engine to execute it.

OPEN WEATHER API

API Open Weather Map is an online service that provides weather data, including current weather data, forecasts, and historical data to the developers of web services and mobile applications. For data sources, it utilizes meteorological broadcast services, raw data from airport weather stations, raw data from radar stations, and raw data from other official weather stations. All data is processed by Open Weather Map in a way that it attempts to provide accurate online weather forecast data and weather maps, such as those for clouds or precipitation. Beyond that, the service is focused on the social aspect by involving weather station owners in connecting to the service and thereby increasing weather data accuracy.

The ideology is inspired by Open Street Map and Wikipedia that make information free and available for everybody. It uses Open Street Map for display of weather maps. Open Weather Map provides an API with JSON, XML and HTML endpoints and a limited free usage tier. Making more than 60 calls per minute requires a paid subscription starting at USD 40 per month. Access to historical data requires a subscription starting at US\$150 per month. Users can request current weather information, extended forecasts and graphical maps (showing cloud cover, wind speed, pressure and precipitation).

Current weather data

Current data is refreshed every ten minutes; it can be searched by city or by geographic coordinates on Earth. Searching The Open Weather Map geocoding system allows users to select cities by name, country, zip-code or geographic coordinates. It is possible to search by part of city name. To make searching result more accurate city name and country should be divided by comma.

CHAPTER – II

Software Requirement Specification

CHAPTER – II

Software Requirement Specification

2.1 Problem Statement

Weather prediction is a useful tool for informing populations of expected weather conditions. Weather prediction is a complex topic and poses significant variation in practice. We will attempt to understand and implement a weather prediction application using the linear regression.

2.2 Project Scope

- Weather forecasts are made by collecting as much data as possible about the current state of the atmosphere (particularly the temperature, humidity and wind) to determine how the atmosphere evolves in the future.
- However, the chaotic nature of the atmosphere makes the forecasts less accurate as the range of the forecast increases.
- Traditional observations made at the surface of atmospheric pressure, temperature, wind speed, wind direction, humidity, precipitation are collected routinely from trained observers, automatic weather stations or buoys. During the data assimilation process, information gained from the observations is used In conjunction with a numerical model's most recent forecast for the time that observations were made to produce the meteorological analysis. The complicated equations which govern how the state of a fluid changes with timerequire supercomputers to solve them.
- The output from this model can be used the weather forecast as alternative.

2.3 Design and Implementation Constraint

- **Technology Limitation:** Restricted to client-side technologies (HTML, CSS, JS) without backend storage or server-side processing.

- **API Dependency:** Relies on OpenWeatherMap API for real-time data.
- **Rate Limiting:** Free-tier API calls are limited per minute/hour/day.
- **Cross-Browser Compatibility:** The app must work consistently across major browsers like Chrome, Firefox, and Edge.
- **No Offline Mode:** Cannot function without an internet connection.

2.4 User Documentation

The product is a general-purpose application software where any user can access the software to gather information about the weather based on a current or a previous data analysis. In this product the software makes a request to the server to access the current dataset through which the data is analyzed using various machine learning algorithms.

2.5 Assumptions and Dependencies

- Assumes users have an internet connection to access the app and API.
- Assumes OpenWeatherMap API is available and functional.
- Application will be used primarily on modern browsers.
- Assumes basic knowledge from users to enter city names correctly.
- No requirement for storing user history or preferences.

2.6 System Features

2.6.1 Actors

Actor	Role Description
User	Enters city name and views weather information
Weather API	Provides real-time weather data upon request
Browser (Client)	Executes frontend code and renders the app to the user

2.7 Functional Requirements

2.7.1 Accessing a Database

- The app does not use a traditional database. Instead, it fetches weather data from the OpenWeatherMap API, which serves as a cloud data source.

2.7.2 Prediction Algorithm

- The application does not implement its own prediction algorithm.
- It relies on OpenWeatherMap's forecasting engine, which provides both current and forecasted weather details.

2.7.3 Actions Performed by the System

- Accept city name input.
- Validate the input (non-empty).
- Send a request to the weather API.
- Parse the JSON response.
- Display:
 - Temperature
 - Humidity
 - Wind Speed
 - Weather description and icon
- Show error message on invalid inputs.

2.8 Non-Functional Requirements

2.8.1 User Non-Functional Requirements

- Usability: The interface must be clean, intuitive, and easy to navigate.
- Accessibility: Should be usable by people with basic digital literacy.
- Responsiveness: Must render properly across devices and screen sizes.

2.8.2 System Non-Functional Requirements

- Performance: API responses must be displayed within 1-3 seconds.
- Reliability: App must gracefully handle failed API calls or connection errors.
- Security: Since no login or sensitive data is handled, minimal security risks.
- Scalability: Not applicable; the app does not maintain sessions or store data.

2.8.3 Other Non-Functional Requirements

- Maintainability: Code should be modular and commented for easy updates.
- Portability: Can be deployed on any static web server (GitHub Pages, Netlify).
- Compliance: Use only publicly available APIs and open web standards (HTML5, CSS3, JS).

2.9 Other Requirements

2.9.1 Performance Requirements

- The application must respond to user input and display weather data within 2–3 seconds after the request.
- The weather data retrieval must not cause any UI lag or freezing.
- The app must be lightweight

2.9.2 Safety Requirements

Since this is a non-critical, browser-based application with no physical hardware or sensitive data involved, safety risks are minimal. However, the following should be ensured:

- Must provide a clear error message if the API fails or the user enters an invalid city.
- Must avoid infinite API request loops that could cause performance issues.

2.9.3 Security Requirements

- All API communications should be done over HTTPS to ensure secure transmission of data.
- The app should not store any user data or require login, reducing the risk of data leaks.
- The OpenWeatherMap API key (if exposed in frontend code) should be rate-limited and not tied to billing or sensitive services.

2.9.4 Hardware Requirements

Component	Minimum Requirement
-----------	---------------------

Processor	Any modern processor (1 GHz or higher)
-----------	--

RAM	Minimum 512 MB RAM
-----	--------------------

Display	800x600 resolution or higher
---------	------------------------------

Internet	Active internet connection (for API access)
----------	---

Input Device	Keyboard and/or touch input
--------------	-----------------------------

2.9.5 Software Requirements

Software Component	Description
--------------------	-------------

Operating System	Windows, Linux, macOS, Android, iOS
------------------	-------------------------------------

Web Browser	Google Chrome, Mozilla Firefox, Edge, Safari
-------------	--

Development Tools	Any text editor (VS Code, Sublime Text, etc.)
-------------------	---

External API	OpenWeatherMap API
--------------	--------------------

Language Support	HTML5, CSS3, JavaScript .
------------------	---------------------------

CHAPTER – III

Design

Chapter III

Design

3. Design

3.1 Introduction

Design is a critical phase in the development lifecycle of any software project. It serves as the bridge between system requirements and implementation. In the context of this Weather App, the design focuses on creating a responsive and intuitive user interface, integrating external API data, and ensuring modular, scalable front-end architecture. The design prioritizes simplicity, user-friendliness, and performance.

3.1.1 Proposed Approach

The proposed approach follows a modular and client-side architecture. The application is entirely developed using front-end technologies: HTML for structure, CSS for styling, and JavaScript for functionality. The app communicates with the OpenWeatherMap API using `fetch()` to retrieve weather data in real-time.

- Lightweight and fast-loading.
- No server-side code or database needed.
- Entire logic runs in the browser.
- API-driven architecture.
- Error handling for robustness.

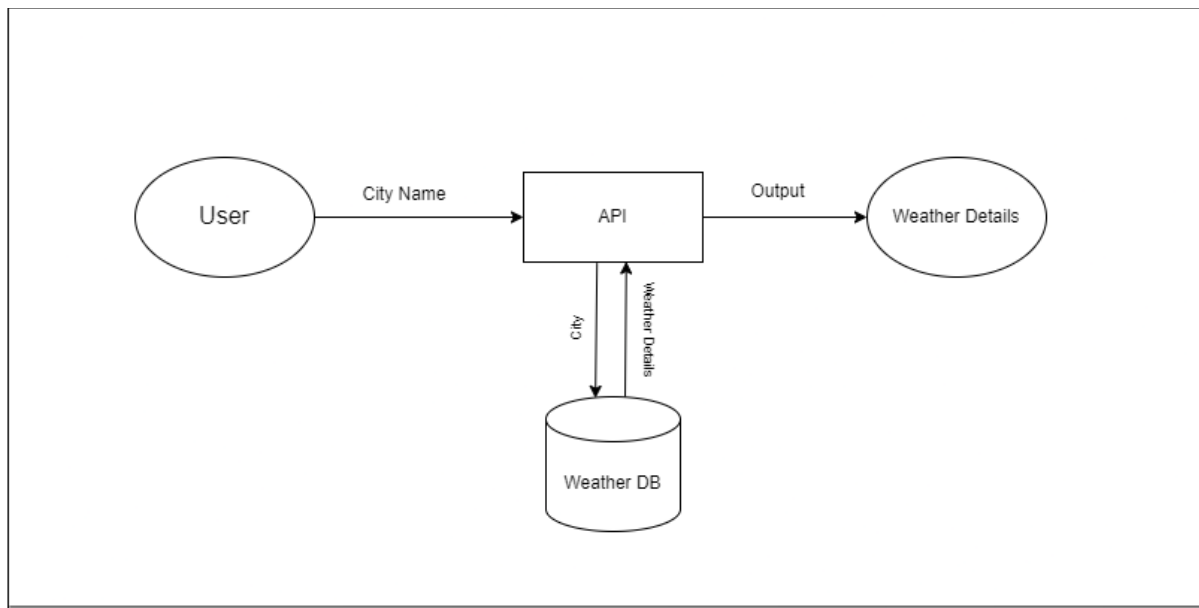
Workflow:

1. User inputs a city name.
2. JavaScript captures input and sends an API request.
3. The API returns JSON weather data.
4. JavaScript processes and displays the data dynamically.

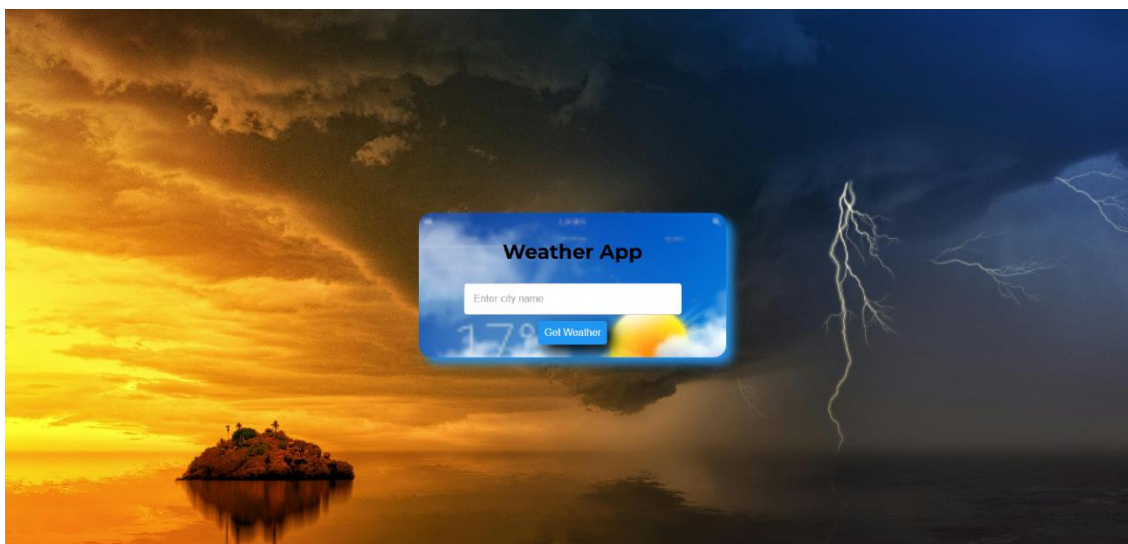
3.1 Use-Case Diagram

An interaction between a user and a system is described by use case diagram. Use case diagrams describe what a system does from the standpoint of an external observer. The emphasis is on what a system does rather than how. Use case diagrams are closely connected to scenarios. A scenario is an example of what happens when someone interacts with the system. A use case diagram is a collection of actors, use cases, and their communications.

For initial development we can use this use case. In this use case diagram we can see following use cases and actor. Use cases are self explanatory and they represent the main functions of Weather Prediction System



3.1 Ui Design



CHAPTER - IV

Testing

Chapter IV

Testing

4. Testing

Testing were critical components of the development process. This phase ensured that each part of the application operated correctly, individually and collectively. Effective testing helped identify bugs early, enhance performance, and improve the overall user experience. Two main types of testing were performed: **Unit Testing** and **Integration Testing**.

● Unit Testing:

Unit testing is the process of testing individual functions or small units of code in isolation to verify that each performs as expected. The goal was to ensure that the key JavaScript functions operated correctly when given various inputs and conditions.

Focus Areas:

- Testing the `fetch()` call to the OpenWeatherMap API with different city names.
- Verifying the correct parsing and formatting of JSON data.
- Ensuring that UI elements (like city name, temperature, weather icon) update accurately.

● Integration Testing:

Integration testing involves testing multiple components together to ensure they interact correctly as a complete system. This verified that the HTML interface, CSS styling, and JavaScript logic worked harmoniously—from capturing user input to rendering real-time weather results.

Focus Areas:

- Ensuring the city input field correctly passes data to the JS function.
- Verifying that JavaScript successfully makes the API request using the provided city name.
- Making sure that the response from the API updates the correct DOM elements (e.g., temperature, city, description, icon).
- Testing the UI behavior when the API fails, such as during network errors or invalid city searches.

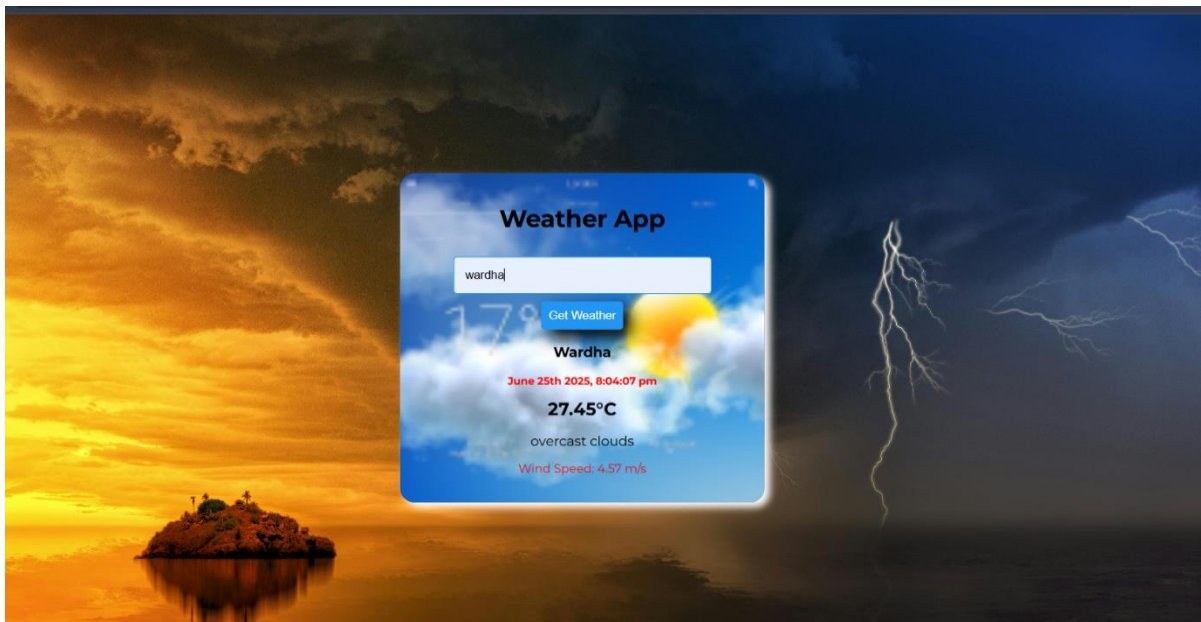
CHAPTER – V

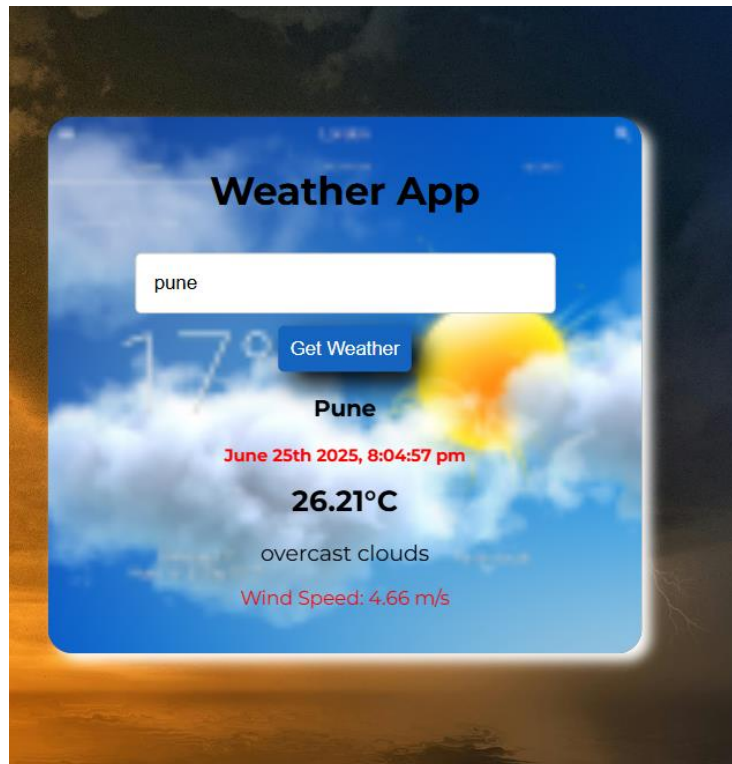
Results

Chapter V

Results

Screenshot





CHAPTER - VI

Conclusion & Future Scope

Chapter VI

Conclusion & Future Scope

Conclusion

The Weather App project was designed and developed using **HTML, CSS, and JavaScript** to help users check the current weather of any city around the world. By using the **OpenWeatherMap API**, the app provides real-time weather information like temperature, weather conditions, and icons in a simple and easy-to-understand format.

This project focused on creating a clean and responsive user interface that works well on both desktop and mobile devices. JavaScript was used to handle user input, send API requests, and update the webpage with weather data. The development process included **unit testing** to check small parts of the code and **integration testing** to ensure all parts of the app work well together.

The app was tested and debugged carefully to make sure it performs correctly in different situations, such as entering an invalid city name or losing internet connection. Overall, the project was a success, and the app meets its goal of helping users quickly access accurate weather information.

Future Scope

Although the Weather App is working well, there are several improvements and new features that can be added in the future to make it more advanced and user-friendly:

- **Add 5-day or weekly weather forecast** so users can plan ahead.
- **Use GPS/location services** to show weather automatically based on the user's current location.
- **Allow temperature unit conversion** between Celsius and Fahrenheit.
- **Add background themes or images** based on weather conditions (e.g., sunny, rainy, snowy).
- **Include weather alerts or warnings** for extreme conditions like storms or heatwaves.
- **Store recent search history** so users can quickly check cities they searched before.
- **Improve accessibility** for visually impaired users by using voice or text-to-speech features.

These future updates will help make the app more powerful, useful, and enjoyable for users.

APPENDIX

A.SOURCE CODE:

HTML:

```
<!DOCTYPE html>

<head>

  <link rel="stylesheet" href="style.css">

  <link rel="stylesheet" href=
"https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css">

  <link rel="stylesheet" href=
"https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.1/css/all.min.css">

  <link rel="stylesheet" href=
"https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&display=swap">

  <title>GFG App</title>

</head>

<body>

  <div class="container">

    <div class="weather-card">

      <h1>

        Weather App

      </h1>

      <input type="text" id="city-input"

        placeholder="Enter city name">

      <button id="city-input-btn"

        onclick="weatherFn($('#city-input').val())">
```

Get Weather

</button>

<div id="weather-info"

class="animate__animated animate__fadeIn">

<h3 id="city-name"></h3>

<p id="date"></p>

<!-- -->

<p id="temperature"></p>

<p id="description"></p>

<p id="wind-speed"></p>

</div>

</div>

</div>

<script src=

"https://code.jquery.com/jquery-3.6.0.min.js">

</script>

<script src=

"https://momentjs.com/downloads/moment.min.js">

</script>

<script src="script.js"></script>

</body>

</html>

CSS:

```
body {
  margin: 0;
  font-family: 'Montserrat', sans-serif;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  background-image: url(pexels-jplenio-1118873.jpg);
  background-size: cover;
}
.container {
  text-align: center;
}
.weather-card {
  background-color: rgba(255, 255, 255, 0.95);
  border-radius: 20px;
  padding: 20px;
  box-shadow: 10px 10px 15px #2196F3;
  transition: transform 0.3s ease-in-out;
  width: 450px;
  background-image: url(bg.png);
  background-size: cover;
  background-repeat: no-repeat;
}
.weather-card:hover {
  transform: scale(1.05);
  box-shadow: 7px 7px 10px #fff;
}
#city-input {
  padding: 15px;
  margin: 10px 0;
  width: 70%;
  border: 1px solid #ccc;
  border-radius: 5px;
  font-size: 16px;
}
#city-input:focus {
  outline: none;
  border-color: #2196F3;
}
```

```
}
#city-input::placeholder {
  color: #aaa;
}
#city-input-btn {
  padding: 10px;
  background-color: #2196F3;
  color: #fff;
  border: none;
  border-radius: 5px;
  font-size: 16px;
  cursor: pointer;
  box-shadow: 10px 10px 20px #000 ;
}
#city-input-btn:hover {
  background-color: #1565C0;
}
#weather-info {
  display: none;
}
#weather-icon {
  width: 100px;
  height: 100px;
}
#temperature {
  font-size: 24px;
  font-weight: bold;
  margin: 8px 0;
}
#description {
  font-size: 18px;
  margin-bottom: 10px;
}
#wind-speed {
  font-size: 16px;
  color: rgb(255, 0, 0);
}
#date {
  font-size: 14px;
  font-weight: 800;
  color: rgb(255, 0, 0);
}
```

```
}
```

JAVASCRIPT:

```
const url =  
'https://api.openweathermap.org/data/2.5/weather';  
const apiKey =  
  '603413aaa71e77ec8da18a831f12b3c3';  
  
$(document).ready(function () {  
  weatherFn('Pune');  
});  
  
async function weatherFn(cName) {  
  const temp =  
    `${url}?q=${cName}&appid=${apiKey}&units=metric`;  
  try {  
    const res = await fetch(temp);  
    const data = await res.json();  
    if (res.ok) {  
      weatherShowFn(data);  
    } else {  
      alert('City not found. Please try again.');    }  
  } catch (error) {  
    console.error('Error fetching weather data:', error);  
  }  
}
```

```
function weatherShowFn(data) {  
  $('#city-name').text(data.name);  
  $('#date').text(moment().  
    format('MMMM Do YYYY, h:mm:ss a'));  
  $('#temperature').  
    html(`${data.main.temp} °C`);  
  $('#description').  
    text(data.weather[0].description);  
  $('#wind-speed').  
    html(`Wind Speed: ${data.wind.speed} m/s`);  
  $('#weather-icon').  
    attr('src',  
      `...`);  
  $('#weather-info').fadeIn();  
}
```

REFERENCES

1. **OpenWeatherMap API**
<https://openweathermap.org/api>
Used to fetch real-time weather data for different cities.
2. **W3Schools – HTML, CSS, JavaScript Tutorials**
<https://www.w3schools.com>
Referred for syntax, examples, and code snippets in web development.
3. **MDN Web Docs (Mozilla Developer Network)**
<https://developer.mozilla.org>
Used for understanding web technologies, JavaScript methods, and browser compatibility.
4. **GeeksforGeeks – JavaScript Programming**
<https://www.geeksforgeeks.org>
Helpful for understanding JavaScript logic and best practices.
5. **Stack Overflow**
<https://stackoverflow.com>
Used to find solutions for coding errors and bugs during development.
6. **Weather Icons from OpenWeatherMap**
Source: <https://openweathermap.org/weather-conditions>
Description: Official weather icons provided by OpenWeatherMap API used to display real-time weather conditions (sun, rain, clouds, etc.).
7. **Background Images (Optional)**
Source: <https://unsplash.com>
Description: High-resolution weather-related images (sunny sky, rainy clouds, snow, etc.) used for dynamic or static backgrounds.